$$\frac{dL}{dw} = \frac{-1}{m}\left[y - \hat{y} \right] x$$

$$\boxed{w = w + \eta \frac{1}{m} (y - \hat{y}) x}$$

~~Precision, Recall, F1 score~~

$$\text{Accuracy score} = \left(\frac{\text{no of } \checkmark}{\text{total pred}} \right)$$

- from sklearn.metrics import accuracy_score
  accuracy_score (y_test, y_pred)

Problem w Accuracy:

A = 90% → 10% incorrect (Accuracy score does not tell us about the type of mistake)

Confusion matrix: → Solves this prob.

from sklearn.metrics import confusion_matrix
confusion_matrix (y_test, y_pred)

Actual \ Predicted	1	0
1	True positive	False negative
0	False positive	True negative

1 → +ve
0 → -ve

1,1 → True
0,0 → True
1,0 → False
0,1

$$\text{Acc-Score} = \frac{TP + TN}{TP + TN + FP + FN}$$

Type 1 error:

False +ve ⟶ (Type 1 error)

False -ve ⟶ (Type 2 error)

Where does accuracy fail:

Imbalanced datasets:

10000

999 — Not terrorist

1 — terrorist

1,1 ⟶ Terrorist that detect hua.

model ⟶ NO one terrorist.

Actual \ Pred	1	0
1	0	1
0	0	999

$$\text{Accuracy} = \frac{999}{999+1} = \frac{99.9.1.}{}$$

Precision:

	Sent to spam	not spam
Spam	100	170
not spam	30	700

M1

100	190
10	700

M2

Accuracy of $M1 = M2$

$FP_1 > FP_2$ (Not spam → Spam) → more dangerous

$FN_2 > FN_1$ (Spam → Not spam)

∴ (False +ve should be minimum)

We deploy model 2

Precision: what proportion of predicted +ve is True positive

TP	FN
FP	TN

$$P = \left(\frac{TP}{TP+FP}\right)$$

$P_{M1} = \frac{100}{130} = 0.76$

$P_{M2} = \frac{100}{110} = 0.99$

$P_{M2} > P_{M1}$

Recall:

	Deleted correctly	Not deleted
Has cancer	1000	200 FN
No cancer	800 FP	8000

90%

	1000	200
	800	8000

90%

FN → Real cancer, not detected → dangerous
FP → No cancer, still detected

$$\text{Recall} = \left(\frac{TP}{TP+FN}\right)$$

Proportion of ves correctly detected.

$$R_A = \frac{1000}{1200}, \quad R_B = \frac{1000}{1500}$$

$$R_A > R_B \ (M_A > M_B)$$

Step 1 → Find which errors are more dangerous
(FP → Type 1, FN → Type 2)

Step 2 → if Type 1 → Precision
Type 2 → Recall.

F1 Score:

$$\text{F1 score} = \frac{2PR}{P+R}$$

P → precision
R → Recall.

(Harmonic mean)

for confusion matrix,

```
pd.DataFrame(confusion_matrix(y_test, y_pred), columns
                                = list(range(0,2)))
```

```
f1_score(y_test, y_pred)
```

```
from sklearn.metrics import classification_report.
classification_report(y_test, y_pred)
```

ROC curve:

Supervised ML

↙ ↘

Regression — y (numerical)

Classification — y (categorical) (1, 0)

In Logistic regression.

iq	cgpa	pred	pred_prob
70	7.0	0	0.43
80	8.0	0	0.39
90	9.0	1	0.61

pred_prob → by sigmoid

$\hat{z} = \ldots$, $\sigma(z)$

Now we determine a threshold,

$\hat{y}$ Th = 0.5,

- pred_prob > 0.5 ⟶ 1.
- " < 0.5 ⟶ 0

★ **But not always 0.5**

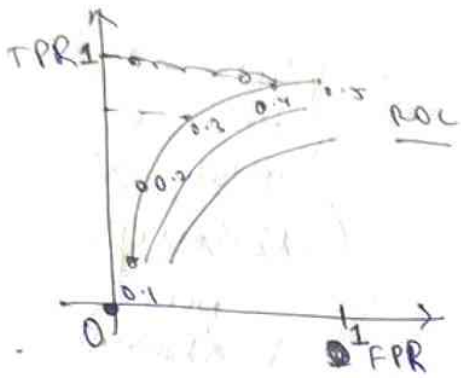
To make a correct decision for threshold, use ROC curve.

ROC → Receiver Operator characteristic

$TPR = \left(\frac{TP}{TP + FN}\right)$, $FPR = \left(\frac{FP}{FP + TN}\right)$

TPR → True +ve Rate

TP	FN
FP	TN

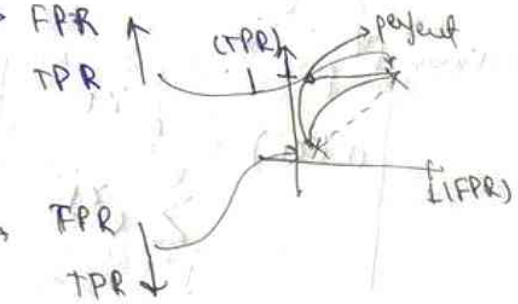


Ideally
TPR = 1
FPR = 0

Point nearest to TPR = 1 is chosen

decrease threshold → FPR ↑, TPR ↑

increase threshold → FPR ↓, TPR ↓



y_scores = model.predict_proba(X_test)[:, 1]

from sklearn.metrics import roc_curve

fpr, tpr, thresholds = roc_curve(y_test, y_pred)

AUC ROC curve: AUC measures the entire 2D Area under neath the ROC curve from (0,0) to (1,1).

AUC = 1 (perfect model)

AUC = 0.5 (random guessing)

AUC = 0 (all classification wrong)